

Representação Intermediária



Universidade Federal do Ceará - Quixadá

Lucas Ismaily
ismailybf@ufc.br

Semestre 2025.1

Compiladores

Baseado nos slides do Prof. Sandro Rigo (IC-Unicamp)

Representação Intermediária (RI)

```
1 class C {  
2     int a; int b; int c;  
3     public void m(){  
4         System.out.println(a+c);  
5         int j = a+b;  
6         String a = "hello";  
7         System.out.println(a);  
8         System.out.println(j);  
9         System.out.println(b);  
10    }  
11 }
```

Representação Intermediária (RI)

```
structure M = struct
  structure E = struct
    val a = 5;
  end
  structure N = struct
    val b = 10
    val a = E.a + b
  end
  structure D = struct
    val d = E.a + N.a
  end
end
```

(a) An example in ML

```
package M;
class E {
  static int a = 5;
}
class N {
  static int b = 10;
  static int a = E.a + b;
}
class D {
  static int d = E.a + N.a;
}
```

(b) An example in Java

- Queremos compiladores para N linguagens, direcionadas para M máquinas diferentes.
- RI nos possibilita elaborar N front-ends e M back-ends, ao invés de $N \cdot M$ compiladores.

Representação Intermediária (RI)

- Facilmente construída a partir da análise semântica.
- Conveniente para a tradução para a linguagem de máquina.
- Facilmente alterada (re-escrita) durante as transformações (otimizações) de código.

Escolha de uma RI

- Reuso: adequação à linguagem e à arquitetura alvo, custos.
- Projeto: nível, estrutura, expressividade.
- “*Intermediate-language design is largely an art, not a science*”, Steven Muchnick.
- Pode-se adotar mais que uma RI.

Características de uma RI

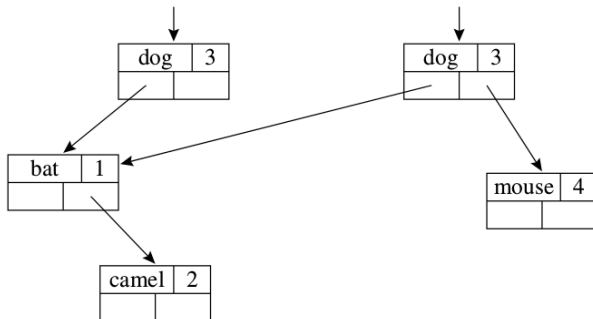
- Alto nível: acesso a arrays, chamada de procedimentos.
- Nível Médio: composta por descrições de operações simples: busca/armazenamento de valores, movimentação, saltos, etc.

Tipos de RI

Representação Gráfica:

- Árvores ou DAGs.
- Manipulação pode ser feita através de gramáticas
- Pode ser tanto de alto-nível como de nível médio.

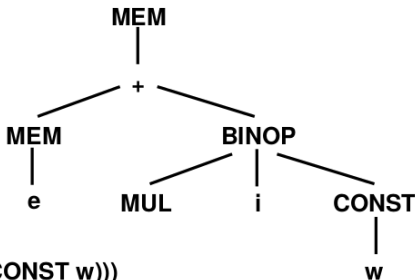
Representação em Árvore vs DAG (RI)



Representação em Árvore

- Tipos de operações (nós): const, binop, mem, call, etc.

Exemplo: $a[i]$



MEM(+ (MEM(e), BINOP(MUL, i, CONST w)))

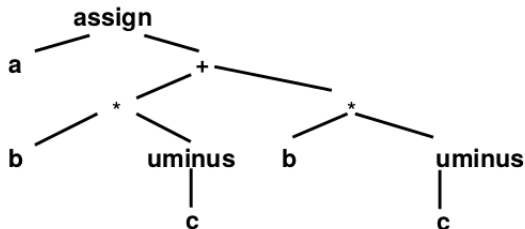
Ref.: Appel, Andrew – Cap 7

Representação Linear

- Representações lineares se assemelham a um pseudo-código para alguma máquina abstrata
 - Java: Bytecode (interpretado ou traduzido).
 - Código de três endereços.

Código de três endereços

- Forma geral $x := y \text{ op } z$
- Representação linearizada de uma árvore sintática ou DAG.



$a := b * -c + b * -c$

```
t1 := - c
t2 := b * t1
t3 := -c
t4 := b * t3
t5 := t2 + t4
a := t5
```

Ref.: Aho, et al – Cap 8

Representação Linear

Tipos de sentenças:

- Atribuição: $x := y \text{ op } z$ ou $z := y$ ou $x := \text{op } y$
- Saltos: goto L
- Desvios condicionais if $x \text{ relop } y$ goto L
- Chamadas a procedimentos: param x and call p,n
- retorno: return y.
- arrays: $x := y[i]$ ou $x[i] := y$

Exemplo: Produto Interno

```
{  
  ...  
  prod = 0;  
  i = 1;  
  while (i <= 20) {  
    prod = prod + a[i] * b[i];  
    i = i + 1;  
  }  
  ...  
}
```



```
(1) prod := 0  
(2) i := 1  
(3) t1 := 4 * i  
(4) t2 := a [ t1]  
(5) t3 := 4 * i  
(6) t4 := b [t3]  
(7) t5 := t2 * t4  
(8) t6 := prod + t5  
(9) prod := t6  
(10) t7 := i + 1  
(11) i := t7  
(12) if i <= 20 goto (3)
```

Representação

■ quádruplas: (op, arg1, arg2, result)

□ (1) $\rightarrow$ (*, b, t1, t2)

a := b * -c + b * -c

■ triplas:

□ (0) $\rightarrow$ (-, c,)

□ (1) $\rightarrow$ (*, b, (0))

□ (2) $\rightarrow$ (-, c,)

(0) t1 := - c

(1) t2 := b * t1

(2) t3 := - c

(3) t4 := b * t3

(4) t5 := t2 + t4

(5) a := t5

Representação

■ Triplas indiretas:

□ (7) $\rightarrow$ (-, c,)

□ (8) $\rightarrow$ (*, b, (7))

□ (9) $\rightarrow$ (-,c,)

■ Lista de sentenças:

□ (0) $\rightarrow$ (7)

□ (1) $\rightarrow$ (8)

□ (2) $\rightarrow$ (9)

□ Array de pointers da a
ordem das triplas no
programa

Geração de código de 3 endereços

- A partir da árvore sintática
- Emissão a partir de ações na gramática com atributos.

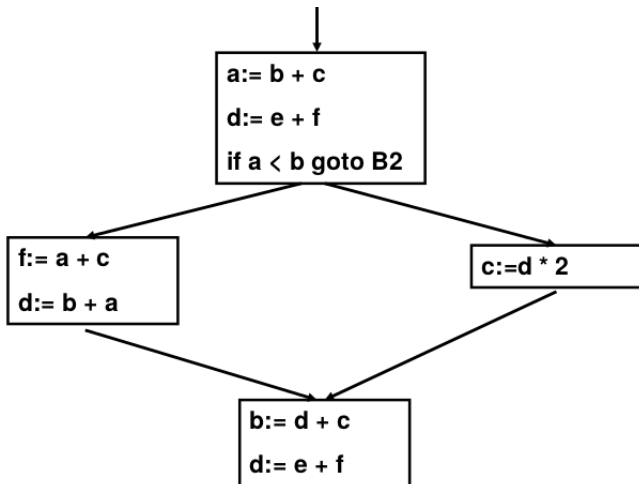
Geração de código de 3 endereços

$S \rightarrow id := E$	$S.code := E.code \parallel gen(id.place \text{ ':=' } E.place)$
$E \rightarrow E1 + E2$	$E.place := newtemp;$ $E.code := E1.code \parallel E2.code \parallel$ $gen((E.place \text{ ':=' } E1.place \text{ '+' } E2.place))$
$S \rightarrow \text{if } E \text{ then } S1$ $\quad \text{else } S2$	$E.true := newlabel; E.false := S.next;$ $S1.next := S.next;$ $S.code := E.code \parallel$ $gen(E.true \text{ ':' }) \parallel S1.code \parallel$ $gen(\text{'goto' } S.next) \parallel gen(E.false \text{ ':' }) \parallel$ $S2.code$

Representação Híbrida

- Combina-se elementos tanto das representações intermediárias gráficas como das lineares.
- Usa RI linear para blocos de código sequencial e uma representação gráfica (grafo CFG) para representar o fluxo de controle entre esses blocos.

Exemplo - CFG com código de três endereços



Caso Real – *Gnu Compiler Collection*

- Várias linguagens: pascal, fortran, C, C++
- Várias arquiteturas alvo: MIPS, SPARC, Intel, Motorola, PowerPC, etc.
- Utiliza mais de uma representação intermediária.
 - Árvore sintática: construída por ações na gramática.
 - RTL: traduções de trechos da árvore.

Caso Real – *Gnu Compiler Collection*

`d := (a+b)*c`

```
(insn 8 6 10 (set (reg:SI 2)
                  (mem:SI (symbol_ref:SI ("a")))))
(insn 10 8 12 (set (reg:SI 3)
                  (mem:SI (symbol_ref:SI ("b")))))
(insn 12 10 14 (set (reg:SI 2)
                  (plus:SI (reg:SI 2) (reg:SI 3))))
(insn 14 12 15 (set (reg:SI 3)
                  (mem:SI (symbol_ref:SI ("c")))))
(insn 15 14 17 (set (reg:SI 2)
                  (mult:SI (reg:SI 2) (reg:SI 3))))
(insn 17 15 19 (set (mem:SI (symbol_ref:SI ("d")))
                  (reg:SI 2)))
```

Caso Real - Xingó

- Representação linear (XIR)
- Utiliza operações que se aproxima das disponíveis na linguagem C.
- Possibilita a geração de código fonte a partir da RI.

Caso Real - Xingó

```
int fat (int n) {  
    if(n==0) return 1;  
    else return n*fat(n-1);}
```

MOVE .t3 n	t3 = n;
MOVEI .t4 0	t4 = 0;
JNE .L2 .t3 .t4	if(t3!=t4) goto L2;
MOVEI .t5 1	t5 = 1;
RET .t5	return t5;
JUMP .L1	goto L1;
LABEL .L2	L2:
MOVE .t7 n	t7 = n;
MOVEI .t8 1	t8 = 1;
SUB .t9 .t7 .t8	t9 = t7 - t8;

Caso Real - Xingó

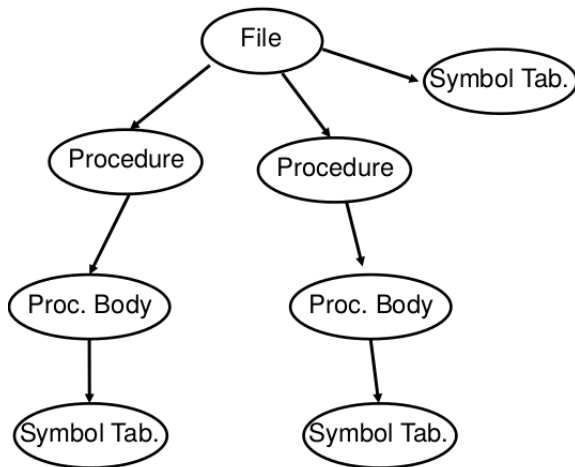
```
CALL [.t10] fat
ARG .t9
MOVE .l1 .t10
MOVE .t13 n
MOVE .t15 .l1
MUL .t16 .t13 .t15
MOVE .l2 .t16
MOVE .t19 .l2
RET .t19
LABEL .L1
```

XIR

```
t10 = fat(t9);
//argumento
l1 = t10;
t13 = n;
t15 = l1;
t16 = t13 * t15
l2 = t16;
t19 = l2;
return t19;
L1;
```

Linguagem C

Caso Real - Xingó



Compilador do Livro do Appel

- O parser produz *abstract syntax trees* (AST).
- Estas árvores refletem a estrutura sintática do programa.
- São a interface entre o parser e as próximas fases da compilação.
- Já abstraem detalhes irrelevantes para as fases seguintes. Ex: pontuação.
- As AST precisam ainda ser transformadas em uma RI mais apropriada para geração de código.

Tradução para RI

- A RI deveria ter componentes descrevendo operações simples
 - Ex.: MOVE, um simples acesso, um JUMP, etc.
- A ideia é quebrar pedaços complicados da AST em um conjunto de operações de máquina.
- Cada operação ainda pode gerar mais de uma instrução assembly no final.

Tradução para RI

- Neste compilador, a RI contém somente corpo das funções.
- Código para o prólogo e o epílogo das funções é adicionado posteriormente.
- A tradução é feita percorrendo a AST e analisando caso a caso.

RI no compilador do Appel

```
package Tree;

abstract class Exp
  CONST(int value)
  NAME(Label label)
  TEMP(Temp.Temp temp)
  BINOP(int binop, Exp left, Exp right)
  MEM(Exp exp)
  CALL(Exp func, ExpList args)
  ESEQ(Stm stm, Exp exp)

abstract class Stm
  MOVE(Exp dst, Exp src)
  EXP(Exp exp)
  JUMP(Exp exp, Temp.LabelList targets)
  CJUMP(int relop, Exp left, Exp right, Label iftrue, Label iffalse)
  SEQ(Stm left, Stm right)
  LABEL(Label label)

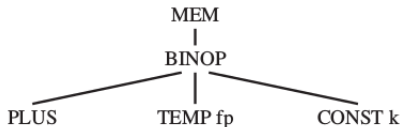
other classes:
  ExpList(Exp head, ExpList tail)
  StmList(Stm head, StmList tail)

other constants:
final static int BINOP.PLUS, BINOP.MINUS, BINOP.MUL, BINOP.DIV, BINOP.AND,
  BINOP.OR, BINOP.LSHIFT, BINOP.RSHIFT, BINOP.ARSHIFT, BINOP.XOR;

final static int CJUMP.EQ, CJUMP.NE, CJUMP.LT, CJUMP.GT, CJUMP.LE,
  CJUMP.GE, CJUMP.ULT, CJUMP.ULE, CJUMP.UGT, CJUMP.UGE;
```

Tradução para RI

Acesso à variáveis:



$\text{MEM}(\text{BINOP}(\text{PLUS}, \text{TEMP } f_p, \text{CONST } k))$

if $x < 5$

$s_1(t, f) = \text{CJUMP}(\text{LT}, x, \text{CONST}(5), t, f)$

Leitura Complementar

- Aho, et al; “Compilers - Principles, Techniques, and Tools”; Cap 8
- Appel, Andrew; “Modern Compiler Implementation in Java”; Cap 7
- A tradução é feita percorrendo a AST e analisando caso a caso.
- Attrot, Wesley; “Xingo - Compilação para uma Representação Intermediária”.
- The Gnu Compiler Collection Reference Manual.

Representação Intermediária



Universidade Federal do Ceará - Quixadá

Lucas Ismaily
ismailybf@ufc.br

Semestre 2025.1

Compiladores

Baseado nos slides do Prof. Sandro Rigo (IC-Unicamp)